



Ingeniería de Software

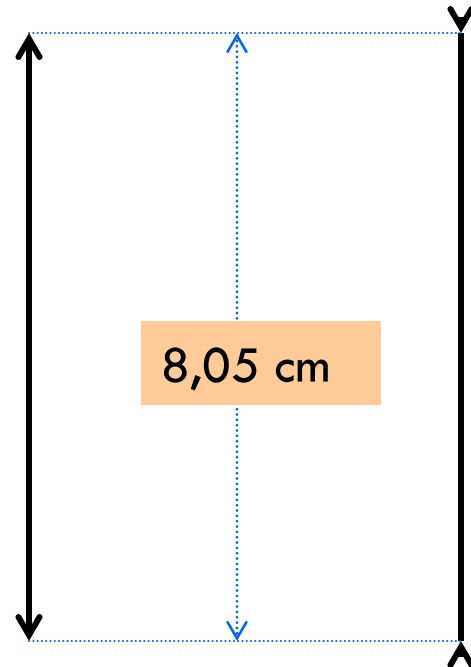


Métricas

Elementos clave de la gestión de proyectos

- ▶ Métricas
 - ▶ Estimaciones
 - ▶ Calendario temporal
 - ▶ Organización del personal
 - ▶ Análisis de riesgos
 - ▶ Seguimiento y control
- Métricas y Estimaciones

¿Cómo saber cuál es mayor?



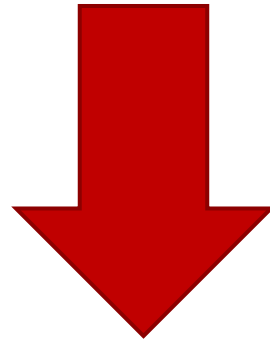
Métricas

- ▶ Las métricas son la clave tecnológica para el desarrollo y mantenimiento exitoso del software. Son un buen medio para entender, monitorear, controlar, predecir y probar el desarrollo software y los proyectos de mantenimiento (Briand et al., 1996)
- ▶ En general, la medición persigue tres objetivos fundamentales (Fenton y Pfleeger, 1997):
 - ▶ entender qué ocurre durante el desarrollo y el mantenimiento
 - ▶ controlar qué es lo que ocurre en nuestros proyectos
 - ▶ mejorar nuestros procesos y nuestros productos



Métricas

Las métricas pueden ser utilizadas para que los profesionales e investigadores puedan tomar las mejores decisiones



Métricas como medio para asegurar la calidad
en los Productos/Procesos/ Proyectos Software

Métricas

- ▶ **¿Para qué realizar mediciones de soft?**
 - ▶ Son la base para realizar buenas estimaciones
 - ▶ Permiten evaluar calidad
 - ▶ Permiten evaluar productividad
 - ▶ Permiten controlar el proyecto
 - ▶ Permiten evaluar beneficios del uso de nuevos métodos
 - ▶



Métricas

- ▶ Nunca deben utilizarse las mediciones para evaluar, premiar o castigar el rendimiento individual



Métricas – Definiciones

- ▶ **Medida:**
 - ▶ indicación cuantitativa de la extensión, cantidad, dimensiones, capacidad o tamaño de algunos atributos de un proceso o producto.
- ▶ **Medición:**
 - ▶ es el acto de determinar una medida.
- ▶ **Métrica:**
 - ▶ medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo dado. El ingeniero de software recopila medidas y desarrolla métricas para obtener indicadores.
- ▶ **Indicador:**
 - ▶ combinación de métricas. Proporciona una visión profunda que permite al gestor de proyectos o a los ingenieros de software ajustar el producto, el proyecto o el proceso para que las cosas salgan mejor

Clasificación de las métricas

▶ Métricas Del Proceso

- ▶ Son aquellas que cuantifican atributos del proceso de ingeniería de software y del equipo de desarrollo.
 - ▶ ¿A qué nos referimos cuando hablamos de proceso?

▶ Métricas Del Producto

- ▶ Son aquellas que se aplican al producto software resultante del proceso de desarrollo de software
 - ▶ Por ej.: tamaño, complejidad



Clasificación de las métricas

▶ Métricas Directas

- ▶ Se obtienen del proceso o producto siendo observado y no dependen de otras medidas.
- ▶ Entre las medidas directas del proceso se encuentran el costo y el esfuerzo, el tiempo de ejecución, el tamaño de memoria requerido, los defectos observados en un período de tiempo.

▶ Métricas Indirectas

- ▶ Dependen de medidas más elementales (directas).
- ▶ Entre ellas se encuentran la funcionalidad, complejidad, eficiencia, fiabilidad, facilidad de mantenimiento.

Clasificación de las métricas

▶ Medidas Objetivas

- ▶ Pueden ser computadas en forma precisa por un algoritmo. Su valor no cambia con el tiempo, lugar u observador.

▶ Medidas Subjetivas

- ▶ Sus valores dependen de apreciaciones personales de quienes las aplican.



Clasificación de las métricas

▶ Métricas privadas

- ▶ Utilizadas individualmente para mejorar el rendimiento
- ▶ “La mejora del proceso puede y debe empezar en el nivel individual” (Humphrey)
 - ▶ Ej.: índice de defectos

▶ Métricas públicas

- ▶ Generalmente asimilan información que originalmente era privada de particulares y equipos

Tipos de Métricas

**Métricas Orientadas
al tamaño**

**Medidas directas del resultado
y del proceso**

**Métricas Orientadas
a la función**

**Medidas indirectas del
software y del proceso**



Métricas orientadas al tamaño

- ▶ Se relacionan con el tamaño de la salida de alguna actividad
- ▶ La medida mas común son las líneas de código fuente entregadas
- ▶ También se usan el número de instrucciones de código objeto entregado y la cantidad de páginas de documentación del sistema

Métricas orientadas al tamaño

► LDC - LÍNEAS DE CÓDIGO

- Medida directa del software y del proceso

Medida discutida porque depende del lenguaje y es post-mortem

- Es para saber en qué tiempo voy a terminar el software y cuántas personas voy a necesitar.
- Es medida directa del software y el proceso por el cual se desarrolla.
- Si una organización de software mantiene registros sencillos, se puede crear una tabla de datos orientados al tamaño

Utilidad de las métricas postmortem

- ▶ Conformar una línea base para futuras estimaciones
- ▶ Ayudar al mantenimiento conociendo la complejidad lógica, tamaño, flujo de información, identificando módulos críticos
- ▶ Ayudar en los procesos de reingeniería

Métricas orientadas al tamaño

- ▶ **Métricas derivadas del proceso de desarrollo:**
 - ▶ Productividad: relación entre KLDC y Persona mes
 - ▶ Calidad: relación entre Errores y KLDC
 - ▶ Costo: relación entre \$ y KLDC

Productividad = $\text{KLDC} / \text{persona-mes}$

Calidad = $\text{errores} / \text{KLDC}$

Documentación = $\text{págs.. Doc.} / \text{KLDC}$

Costo = $\text{\$/KLDC}$

KLDC (miles de líneas de código)



Métricas orientadas al tamaño

- ▶ LDC - LÍNEAS DE CÓDIGO
- ▶ Exigen explicar el manejo de:
 - ▶ líneas en blanco, líneas de comentarios , declaraciones de datos, líneas con varias instrucciones separadas
 - ▶ Sino...



Información que se pierde: espacio que ocupa en disco, páginas que requiere el listado.



Propuesta Fenton/Pfleeger

- ▶ **Medir :**
 - ▶ CLOC = Cantidad de líneas de comentarios
- ▶ **Luego:**
 - ▶ long total (LOC) = NCLOC + CLOC
- ▶ **Surgen medidas indirectas:**
 - ▶ CLOC/LOC mide la densidad de comentarios



Ejemplo

- ▶ Calcular, usando LDC , la productividad, calidad y costo para los cuatro proyectos de los cuales se proporcionan los datos.

Proyecto	LDC	U\$S	Errores	Personas-mes
P1	25.500	15000	567	15
P2	19.100	7200	210	10
P3	10.700	6000	100	20
P4	100.000	18000	2200	30

- ▶ ¿Cuál es el proyecto de mayor calidad (errores/KLDC)?
- ▶ ¿Cuál es el proyecto de mayor costo por línea (\$/KLDC)?
- ▶ ¿Cuál es el proyecto de menor productividad por persona (KLDC/personas-mes)?



Ejemplo

Proyecto	LDC	U\$S	Error es	Persona s-mes	Errores/K LDC	U\$S/KLD C	KLDC/Per sonas-mes
P1	25.500	15000	567	15	22,23	588,23	1,7
P2	19.100	7200	210	10	10,99	376,96	1,91
P3	10.700	6000	100	20	9,34	560,74	0,53
P4	100.000	18000	2200	30	22	180	3,33



Métricas orientadas a la función

- ▶ Están relacionadas con la funcionalidad total del software entregado.
- ▶ La productividad se expresa según la cantidad de funcionalidad útil producida en un tiempo dado



Métricas orientadas a la función

► Puntos De Función

- Puntos de función (FP): miden la cantidad de funcionalidad de un sistema descrito en una especificación



Métricas orientadas a la función

PF- Punto función (Albrecht 1978)

Factor de Ponderación, es subjetivo y esta dado por la organización/equipo

$$PF = TOTAL * [0.65 + 0.01 * \sum_{i=1}^{14} F_i] \quad 0 \leq F_i \leq 5$$

	simple	medio	complejo	
# Entradas	* [3 4 6]	=		
# Salidas	* [4 5 7]	=		
# Consultas	* [3 4 6]	=		
# Almacenamientos internos	* [7 10 15]	=		
# Interfaces externas	* [5 7 10]	=		
				TOTAL

Son valores de ajuste de la complejidad según las preguntas de la siguiente pantalla

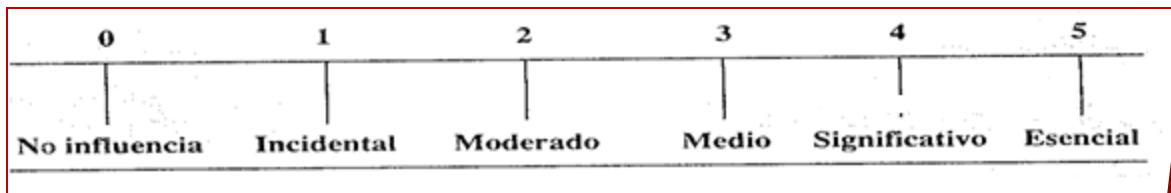
Medida subjetiva independiente del lenguaje, de estimación más fácil. Métrica temprana



Métricas orientadas a la función:

Punto función

1. ¿Requiere el sistema copias de seguridad y de recuperación fiables?
2. ¿Se requiere comunicación de datos?
3. Existen funciones de procesamiento distribuido?
4. ¿Es crítico el rendimiento?
5. ¿Se ejecuta el sistema en un entorno operativo existente y fuertemente utilizado?
6. ¿Requiere el sistema entrada de datos interactiva?
7. ¿Requiere la entrada de datos interactiva que las transacciones de entrada se lleven a cabo sobre múltiples pantallas u operaciones?
8. ¿Se actualizan los archivos maestros de forma interactiva?
9. ¿Son complejas las entradas, las salidas, los archivos o las peticiones?
10. ¿Es complejo el procesamiento interno?
11. ¿Se ha diseñado el código para ser reutilizable?
12. ¿Están incluidas en el diseño la conversión y la instalación'?
13. ¿Se ha diseñado el sistema para soportar múltiples instalaciones en diferentes organizaciones?
14. ¿Se ha diseñado la aplicación para facilitar los cambios y para ser fácilmente utilizada por el usuario?



Cada una de las preguntas se contesta de acuerdo a la siguiente escala de valores

Métricas orientadas a la función:

Punto función

- ▶ Métricas derivadas:
- ▶ Productividad: relación entre PF y Persona_mes
- ▶ Calidad: relación entre Errores y PF
- ▶ Costo: relación entre \$ y PF

$$\begin{aligned}\textbf{Productividad} &= \text{PF} / \text{Persona_mes} \\ \textbf{Calidad} &= \text{Errores} / \text{PF} \\ \textbf{Costo} &= \$ / \text{PF}\end{aligned}$$

Métricas orientadas a la función: Punto característica

Punto característica

- # Entradas
- # Salidas
- # Consultas
- # Almacenamientos internos
- # Interfaces externas

+

Algoritmos



Métricas orientadas a la función

► Punto de característica

- La medida del punto de característica se utiliza en aplicaciones cuya complejidad algorítmica es alta. Para calcular los puntos de características, nuevamente se cuentan y ponderan los valores de E/.S/.. etc. Además las métricas de punto de característica tienen en cuenta otra característica del software, los algoritmos.

Un algoritmo se define como un problema de complejidad computacional limitada que se incluye dentro de un determinado programa de computadora.



Métricas orientadas a la función

► Punto de característica

- Para calcular los puntos de característica, se utiliza la siguiente tabla.

FACTOR DE PONDERACIÓN				
Parámetro de medición	Cuenta		Complejo	
Numero de entradas de usuario		X	4	=
Numero de salidas de usuario		X	5	=
Numero de peticiones de usuario		X	4	=
Numero de archivos		X	7	=
Numero de interfaces externas		X	7	=
Algoritmos		X	3	=
Cuenta = Total				→

Métricas orientadas a la función

▶ Punto de característica

- ▶ Se usa un único valor de peso para cada uno de los parámetros de medida y se calcula el valor del punto característica global mediante la ecuación.
- ▶ $PF = CUENTA_TOTAL * [0.65 + 0.01 * SUM(fi)]$
- ▶ Debe tenerse en cuenta que los puntos de característica y los puntos de función representan lo mismo: "funcionalidad o utilidad" en forma de software.



Reconciliación de las métricas de LDC y PF

Lenguaje	LDC/PF
Ensamblador	320
C	128
Cobol	105
Fortran	105
Pascal	90
ADA	70
C++	64
Smalltalk	22
Generadores de Código	14
SQL	12

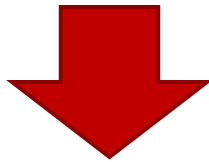
La tabla siguiente proporciona estimaciones informales del número medio de líneas de código que se requiere para construir un punto de función en varios lenguajes de programación:

Depende del lenguaje de programación que se utilice para implementar el software. Ejemplo: 1 línea de código ADA proporciona 1,4 veces más de funcionalidad que una de FORTRAN

Reconciliación De Las Distintas Métricas

► Ejemplo :

- Proyecto “A” LDC : 19.100 escrito en Pascal

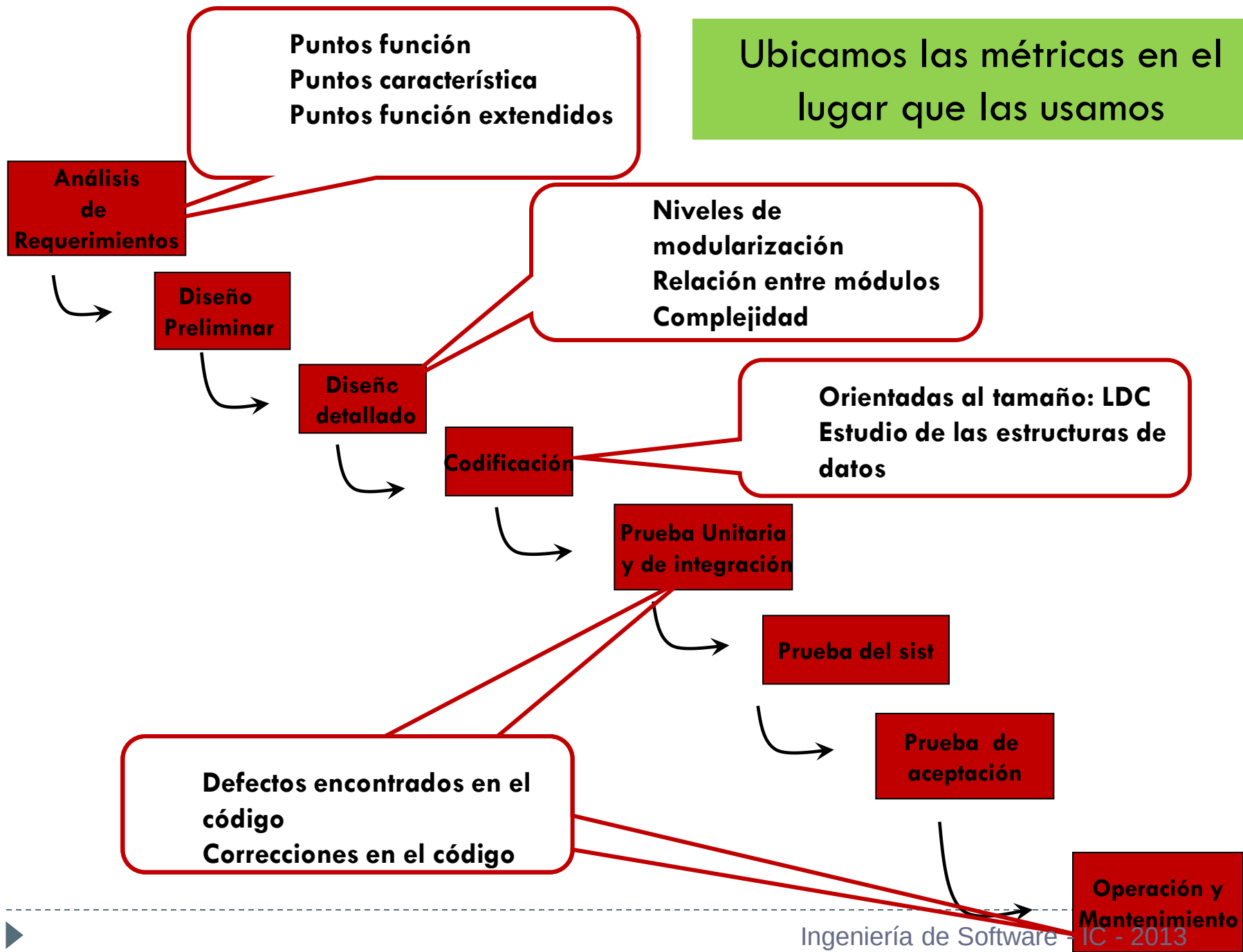


$$\begin{aligned} \text{LDC}/\text{PF} &= 90 \\ \text{PF} &= \text{LDC}/90 \\ \text{PF} &= 19100/90 \end{aligned}$$

$$\text{PF} = 212,2$$



Ubicamos las métricas en el lugar que las usamos





Estimaciones

Estimaciones

- ▶ Estimación: Técnicas que permiten dar un valor aproximado.
- ▶ Para obtener estimaciones confiables generalmente se usan varias técnicas y se comparan y concilian resultados.
- ▶ La estimación no es una ciencia exacta.
- ▶ Modificaciones en la especificación hacen peligrar las estimaciones.
- ▶ Requiere experiencia, acceso a información histórica y decisión para convertir información cualitativa en cuantitativa.



Estimaciones

El riesgo de la estimación decrece con la disponibilidad de historia

- Se realizan estimaciones de recursos, costos y tiempos
- Los factores que influyen son la complejidad, el tamaño, la estructuración del proyecto.



Estimaciones de recursos

- ▶ Recursos humanos
- ▶ Recursos de software reutilizables
- ▶ Recursos de hard y herramientas de software

- ▶ Cada recurso requiere:
 - ▶ Descripción
 - ▶ Informe de disponibilidad
 - ▶ Fecha en que se lo requiere
 - ▶ Tiempo que se lo necesita



Técnicas de estimación

- ▶ **Juicio experto:**

- ▶ Se consultan varios expertos. Cada uno de ellos estima. Se comparan y discuten

- ▶ **Técnica Delphi:**

- ▶ Consiste en la selección de un grupo de expertos a los que se les pregunta su opinión. Las estimaciones de los expertos se realizan en sucesivas rondas, anónimas, con el objeto de tratar de conseguir consenso, pero con la máxima autonomía por parte de los participantes.

- ▶ **División de trabajo:**

- ▶ Jerárquica hacia arriba



Modelos empíricos de estimación

- ▶ Utilizan fórmulas derivadas empíricamente para predecir costos o esfuerzo requerido en el desarrollo del proyecto.

Ej: MODELO COCOMO de Boehm (1981)
(COnstructive COst MOdel, modelo constructivo de costos)
se obtuvo recopilando datos de varios proyectos grandes.

- ▶ Las formulas que utiliza el COCOMO vinculan el tamaño del sistema y del producto, factores del proyecto y del equipo con el esfuerzo necesario para desarrollar el sistema.

Estimaciones: COCOMO 81

El modelo inicial consideraba tres tipos de proyectos:

Orgánicos: Proyectos pequeños y de poca gente

Semiacoplados: Proyectos intermedios

Empotrados: Proyectos con restricciones rígidas

Modelo COCOMO

El tipo BÁSICO estima a través de LDC

Tipos de proyectos: Orgánicos - Semiacoplados - Empotrados

a	2.4	3.0	3.6
b	1.05	1.12	1.20

$E = a (KLDC)^b$ Esfuerzo en persona-mes

Fórmula básica de nivel I de estimación, había tres niveles y profundizaban el detalle de la estimación.

También suponía un modelo de proceso en cascada con uso de lenguajes imperativos del estilo “C” o fortran



Estimaciones

- ▶ Ejemplo: Suponer que se cuenta con un Proyecto con:
LDC : 19.100 y Semiacoplado
- ▶ $E = 3 * (19,1) 1,12 = 81,63$ esfuerzo en personas/mes
- ▶ $D = 2,5 * E 0,35 = 11,67$ meses

Formula de nivel I para
el calculo de duración
en meses

Modelo COCOMO

Tipos de proyectos: Orgánicos - Semiacoplados - Empotrados

c	2.5	2.5	2.5
d	0.38	0.35	0.32

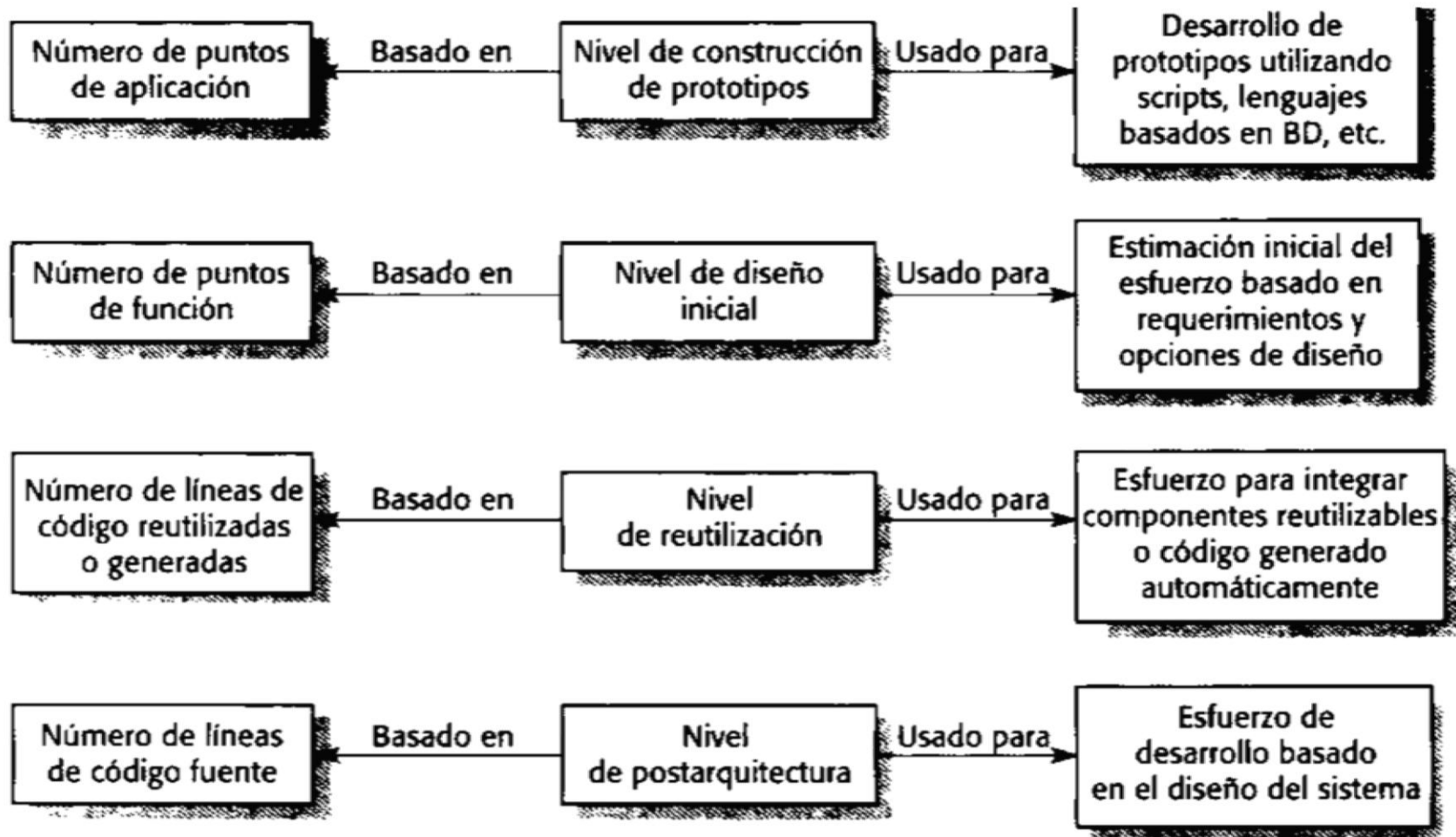
$D = cE^d$ Duración en meses

COCOMO II (2000)

- ▶ Reconoce que las líneas de código son difíciles de estimar tempranamente. Considera diferentes enfoques para el desarrollo , como la construcción de prototipos, el desarrollo basado en componentes, desarrollo en espiral y engloba varios niveles que producen estimaciones detalladas de forma incremental.
- ▶ COCOMO II está compuesto por 4 niveles:
 - ▶ De construcción de prototipos
 - ▶ De diseño inicial
 - ▶ De reutilización
 - ▶ De post-arquitectura

COCOMO II

Niveles de COCOMO II



COCOMO II

Nivel Construcción de prototipos

La fórmula para el cálculo del esfuerzo para el prototipado del sistema es:

Punto de aplicación = Punto
objeto

$$PM = (NAP \times (1 - \%reutilización / 100)) \times PROD$$

PM = esfuerzo estimado en personas/mes

NAP = total de puntos de aplicación en el sistema a desarrollar.

PROD = productividad medida en puntos objeto

COCOMO II

1. Nivel de construcción de prototipos

Productividad de puntos de objeto, se basa en la siguiente tabla para obtener el PROD de la formula anterior

Experiencia y capacidad de los desarrolladores	Muy baja	Baja	Media	Alta	Muy alta
Madurez y capacidad de las herramientas CASE	Muy baja	Baja	Media	Alta	Muy alta
PROD (NOP/mes)	4	7	13	25	50

COCOMO II

2. Nivel de diseño inicial

La fórmula para las estimaciones en este nivel es:

$$\text{Esfuerzo} = A \times \text{Tamaño} \times B \times M.$$

Utilizando la tabla de
conversión

Boehm propone que $A = 2.94$. (Otros autores proponen: 2.45)

Tamaño = KLDC (miles de líneas de código fuente).

$B = 0.91 \times \sum SF_j$ (ver tabla SF) Corrige según ambiente; < 1 bueno al crecer; > 1 malo al crecer

$M = \text{PERS} \times \text{RCPX} \times \text{RUSE} \times \text{PDIF} \times \text{PREX} \times \text{FCIL} \times \text{SCED}$
(ver tabla)

COCOMO II - 2. Nivel de diseño inicial

Tabla de Valores de escala SF para calcular B

	Muy bajo	bajo	nominal	alto	Muy alto	Extra alto
PREC	6.20	4.96	3.72	2.48	1.24	0.00
FLEX	5.07	4.05	3.04	2.03	1.01	0.00
RESL	7.07	5.65	4.24	2.83	1.41	0.00
TEAM	5.48	4.38	3.29	2.19	1.10	0.00
PMAT	7.80	6.24	4.68	3.12	1.56	0.00

PREC: experiencia de proyectos precedentes. Desde totalmente sin precedentes hasta totalmente familiar

FLEX: flexibilidad de desarrollo. Desde requerimientos muy rígidos hasta solamente metas generales

RESL: Nivel de riesgos y tiempo dedicado a arquitectura, desde casi nada a total

TEAM: cohesión del equipo. Desde dificultades graves en interacción hasta interacción suave.

PMAT: Madurez de acuerdo a CMM: Desde 1 hasta 5 (uno se repite)

COCOMO II – 2. Nivel de diseño inicial

Aclaración siglas de M

M: Multiplicador.

Son siete características/atributos del proyecto y del proceso que influyen en la estimación. Éstas hacen que aumente o disminuya el esfuerzo requerido.

- **RCPX** = Fiabilidad y complejidad del producto.
- **RUSE** = Reutilización requerida.
- **PDIF** = Dificultad de la plataforma.
- **PERS** = Capacidad del personal.
- **PREX** = Experiencia del personal.
- **SCED** = Calendario.
- **FCIL** = Facilidades de apoyo.

Se pueden estimar directamente en una escala de 1 (valor muy bajo) a 6 (valor muy alto).

COCOMO II

Nivel 3: reutilización

Para el código generado automáticamente, el modelo estima el número de persona/mes necesarias para integrar este código.

$$\mathbf{PM_{auto} = (ASLOC \times AT/100)/ATPROD.}$$

AT = porcentaje de código adaptado que se genera automáticamente.

ATPROD = productividad de los ingenieros que integran el código

ASLOC = Nro de líneas de código en los componentes que deben ser adaptadas

COCOMO II

Nivel 4: post-arquitectura

- ▶ Las estimaciones están basadas en la misma fórmula básica

$$\text{▶ } \mathbf{PM = A \times Tamaño^B \times M}$$

- ▶ pero se utiliza un conjunto más extenso de atributos (17 en lugar de 7) de producto, proceso y organización para refinar el cálculo del esfuerzo inicial.



COCOMO II

Nivel 4: post-arquitectura

- ▶ La estimación del número de líneas de código se calcula utilizando tres componentes:
 - ▶ Una estimación del número total de líneas nuevas de código a desarrollar.
 - ▶ Una estimación del número de líneas de código fuente equivalentes (ESLOC) calculadas usando el nivel de reutilización.
 - ▶ Una estimación del número de líneas de código que tienen que modificarse debido a cambios en los requerimientos.
- ▶ Estas estimaciones se añaden para obtener el tamaño del código (KLDC).



COCOMO II

Nivel 4: post-arquitectura

- ▶ Estas estimaciones se añaden para obtener el tamaño del código (KLDC).
- ▶ El exponente B se calcula considerando 5 factores de escala.

Como

- Productividad de desarrollo
- Cohesión del equipo
- Entre otros

Estimaciones on-line

- ▶ Visitar las páginas:

- ▶ Para COCOMO8I:

- ▶ http://sunset.usc.edu/research/COCOMOII/cocomo8I_pgm/cocomo8I.html

- ▶ Para COCOMO 2:

- ▶ <http://csse.usc.edu/tools/COCOMOII.php>



El desarrollo de una métrica

- ▶ Victor Basili desarrolló un método llamado GQM (Goal, Question, Metric) (o en castellano: OPM Objetivo, Pregunta, Métrica).
- ▶ (<ftp://ftp.cs.umd.edu/pub/sel/papers/gqm.pdf>)
- ▶ Dicho método esta orientado a lograr una métrica que “mida” cierto objetivo . El mismo nos permite mejorar la calidad de nuestro proyecto.

